

Генерация и прохождение больших лабиринтов на Python

Зачётная работа по Информатике

Выполнил: [Мехоношин Марк Антонович]

Группа: [РПО 25/4]

Актуальность и цель работы

- ▶ Актуальность:
 - ▶ • Задача поиска пути — классическая задача теории графов
 - ▶ • Применяется в: робототехнике, компьютерных играх, GPS-навигаторах
- ▶ Цель работы:
 - ▶ • Разработать модуль для генерации больших лабиринтов
 - ▶ • Сравнить эффективность алгоритмов поиска пути (A^* и DFS)

Поставленные задачи

- ▶ Генерация лабиринта с гарантированным путём
- ▶ Реализация алгоритма поиска A*
- ▶ Реализация алгоритма поиска DFS
- ▶ Визуализация лабиринта и пути (matplotlib)
- ▶ Интерактивный режим (консольная игра WASD)
- ▶ Сравнительный анализ алгоритмов

Используемые алгоритмы

- ▶ Генерация лабиринта (Recursive Backtracker / DFS):
 - Создаёт «идеальный» лабиринт (без циклов и изолированных зон)
 - Время работы: $O(N)$ — линейное от количества клеток
- ▶ Поиск пути — A* (A-Star):
 - Использует манхэттенскую эвристику (знает направление к цели)
 - Гарантирует нахождение кратчайшего пути
 - Время работы: $O(b^d)$, но на практике очень быстрый
- ▶ Поиск пути — DFS (альтернатива):
 - Идёт до упора, потом возвращается
 - Быстрее, но не гарантирует кратчайший путь

Структура программы

► MazeGenerator

► (генерация)

►

► • generate()

► • display()

►

•
get_stats()
get_neighbors()

► ☒ Модульность — легко расширять и тестировать

► ☒ Разделение генерации и поиска

MazeSolver

(поиск)

• solve_astar()

• solve_dfs()

Сравнение алгоритмов поиска

Характеристика	A*	DFS
Тип	Информированный	Неинформированный
Кратчайший путь	✓ Да	✗ Нет
Скорость (51×51)	0.03 сек	0.02 сек
Использует эвристику	✓ Да	✗ Нет
Требуется память	Средняя	Малая

Результаты тестирования

Размер	Клеток	Время генер.	Длина пути	Время поиска A*
21×21	441	0.01 сек	47 шагов	0.01 сек
51×51	2 601	0.05 сек	187 шагов	0.03 сек
101×101	10 201	0.21 сек	423 шага	0.12 сек
151×151	22 801	0.48 сек	689 шагов	0.28 сек

Интерактивный режим (консольная игра)

- ▶ Управление:

- ▶ W / ↑ — вверх S / ↓ — вниз
- ▶ A / ← — влево D / → — вправо
- ▶ Q — выход из игры

- ▶ Обозначения в игре:

- ▶  — игрок  — выход
- ▶  — стена
- ▶ (пробел) — коридор

- ▶ Плюсы консольной версии:

- ▶ • Не требует установки дополнительных библиотек
- ▶ • Работает на любом компьютере с Python

Возможности программы

- ▶ ☒ Генерация больших лабиринтов (до 151×151 клеток)
- ▶ ☒ Автоматический поиск пути (A* — оптимальный маршрут)
- ▶ ☒ Альтернативный поиск пути (DFS — для сравнения)
- ▶ ☒ Интерактивное прохождение (WASD, игра самому)
- ▶ ☒ Визуализация через matplotlib (цветная графика)
- ▶ ☒ Консольная версия (без внешних библиотек)
- ▶ ☒ Подробная статистика (стены/проходы, длина пути)
- ▶ ☒ Сравнение эффективности алгоритмов

Итоги работы

- ▶ Что сделано:
- ▶ 1. Разработан модуль генерации лабиринтов (DFS-бэктрекинг)
- ▶ 2. Реализованы 2 алгоритма поиска пути (A* и DFS)
- ▶ 3. Создана визуализация с помощью matplotlib
- ▶ 4. Добавлен интерактивный режим (игра с управлением WASD)
- ▶ 5. Проведено тестирование на разных размерах (до 151×151)
- ▶ Результат:
- ▶ • Программа успешно генерирует большие лабиринты
- ▶ • A* находит кратчайший путь за доли секунды
- ▶ • Возможность как автоматического, так и ручного прохождения

Заключение и перспективы

- ▶ Достигнутые цели:
 - ▶ ✓ Создан полностью рабочий программный продукт
 - ▶ ✓ Проведён сравнительный анализ алгоритмов (A^* vs DFS)
 - ▶ ✓ Реализованы 2 режима работы (авто и интерактив)
- ▶ Перспективы развития:
 - ▶ • Добавление других алгоритмов (Дейкстра, BFS, Bellman-Ford)
 - ▶ • Графический интерфейс (Tkinter / PyQt)
 - ▶ • 3D-визуализация лабиринта
 - ▶ • Генерация лабиринтов нестандартных форм

Спасибо за внимание!

Репозиторий проекта:

<https://github.com/sadsadsadsad31/inf-proekt>